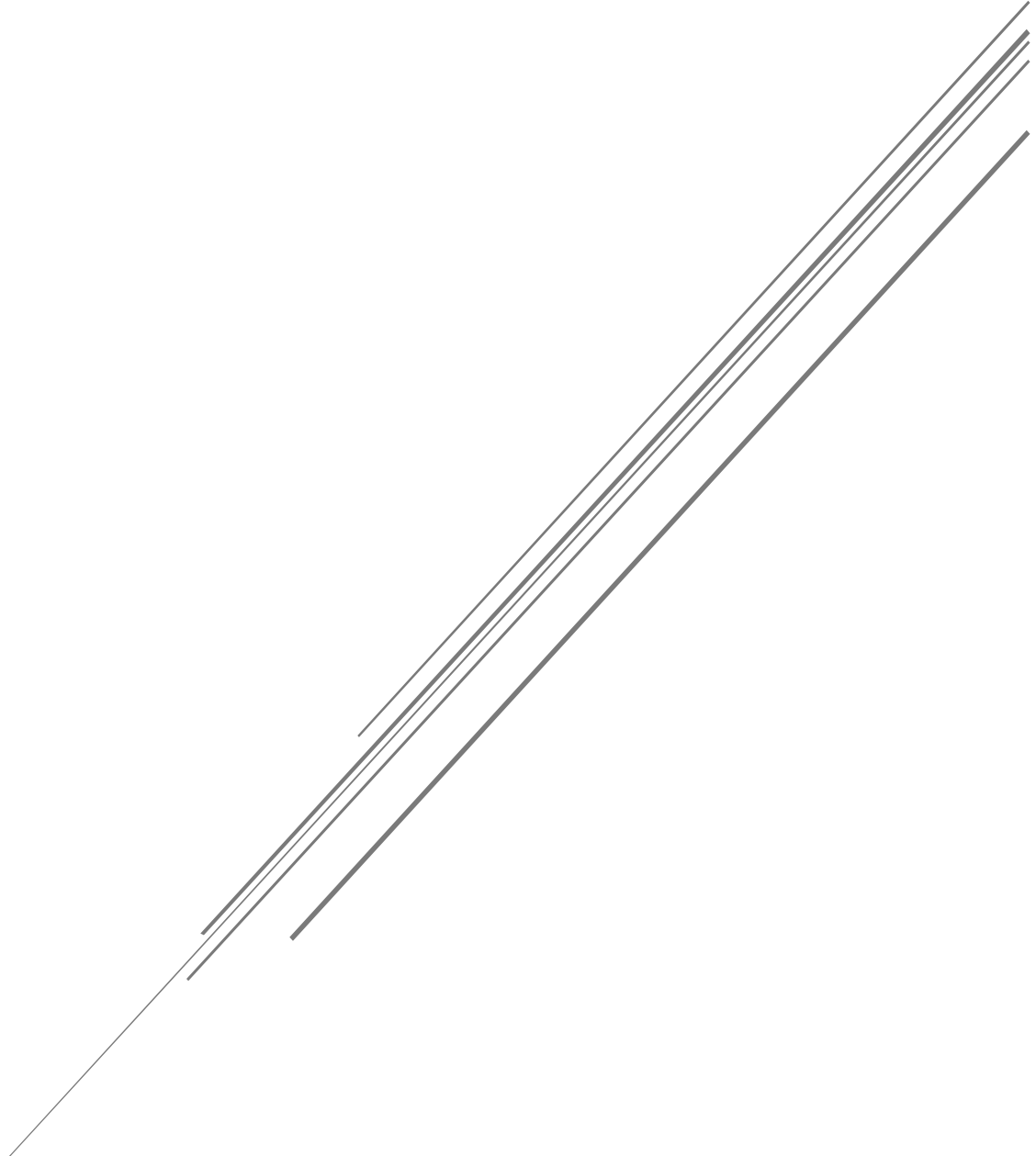


RELATÓRIO DO PROJETO FINAL

Website de registo de avarias



SEGURANÇA NO DESENVOLVIMENTO DE SOFTWARE
Projeto Final

Autor: Ana Margarida Diogo Santos

Informações Gerais

IDE: Visual Studio Code

Extensão: SonarQube for IDE

Desenvolvimento do website iniciado na Tarefa 5.

Início do Projeto

Este projeto tem como ponto de partida o ficheiro index.html criado na sessão 5.

O código inicial foi dividido em 3 ficheiros, de acordo com a linguagem utilizada. Assim, o ficheiro index.html ficou com o código html para interação com o utilizador, o ficheiro style.css ficou com o código css para formatação e o ficheiro login.php tem o código php com as funções de tratamento de dados de utilizadores e informações do login.

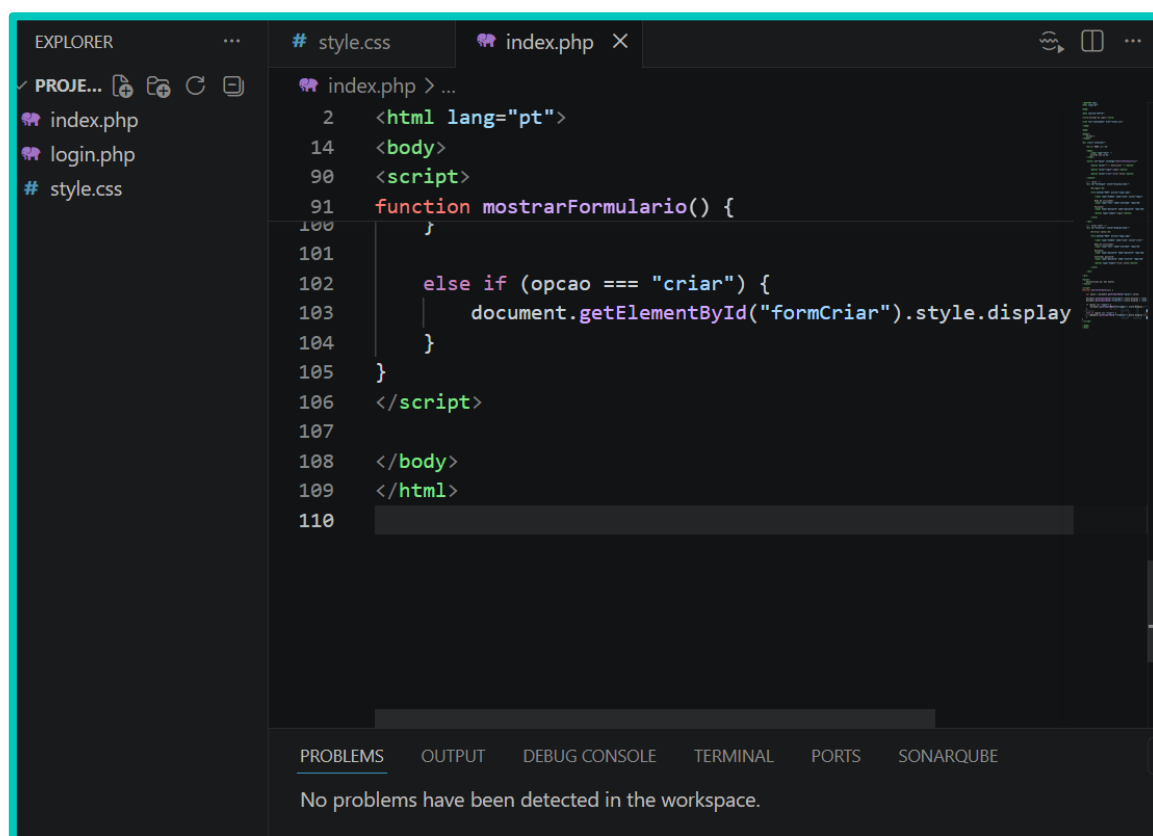


Figura 1

O ficheiro inicial já continha codificação de password: BCrypt, verificação de critérios de password, tratamento de dados sensíveis em funções, utilização de lista de opção em vez de caixa de texto simples.

Os dados de login são enviados através do método POST por ser mais seguro.

Base de Dados

Para permitir o registo permanente dos dados optei por integrar uma base de dados no website. Para aumentar a segurança do website, a ligação à base de dados é feita através de um ficheiro independente, ligabd.php (figura 2).

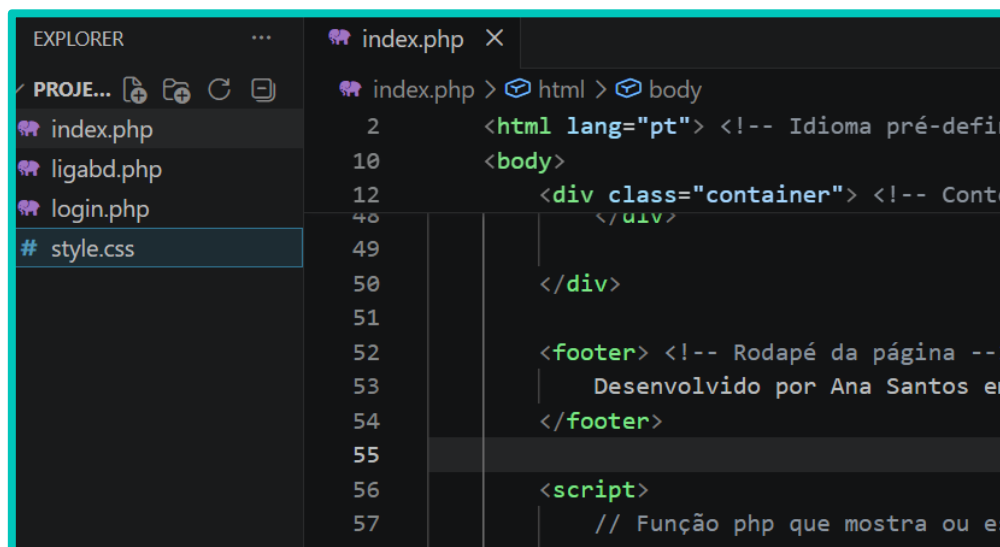


Figura 2

Análise de código – SonarQube

Todos os ficheiros do projeto foram testados com o SonarQube em execução e foram corrigidos todos os alertas, com exceção do aviso de segurança de password no ficheiro ligabd.php por se tratar de uma ligação local (figura 3). Este aviso poderia ser eliminado, através da criação de um outro utilizador administrador da base de dados ou da criação de uma password para o utilizador root.

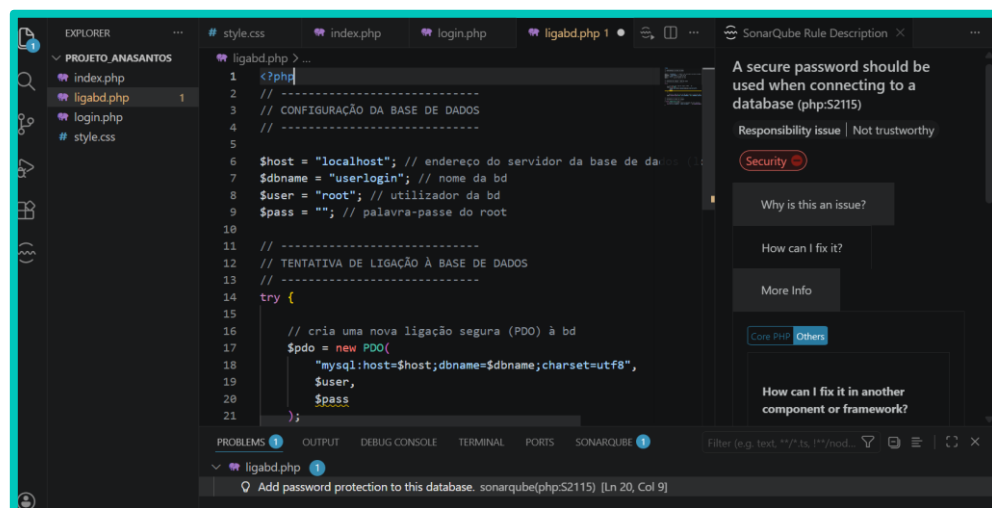


Figura 3

Utilização de sessões

Para que o sistema se “lembre” do utilizador entre páginas e pedidos, sem ele ter de fazer login a cada ação, implementei o sistema de sessões.

O ficheiro logout.php garante que a sessão é destruída no final.

Redireccionamento de utilizadores

Para permitir uma melhor gestão do website e para evitar que o administrador tenha que aceder diretamente à base de dados para gerir os utilizadores e os tickets, alterei o código para ter dois níveis de utilizadores (user ou admin). Consoante o tipo de utilizador a página de redireccionamento é diferente.

O nível user é direcionado para a página inconformidades.php onde pode reportar uma avaria cujos dados são enviados para a página avarias.php. As informações são inseridas através de lista de opções e uma caixa de texto com restrição de caracteres.

O nível admin é direcionado para a página dashboard.php onde tem um menu de gestão.

No menu de gestão do admin há uma opção Tickets que direciona para a página avarias.php.

A página avarias.php recebe as inconformidades registadas pelos utilizadores e permite ao administrador validar os tickets e assinalar quando são resolvidos.

Reflexão

Este projeto simula a implementação de um website de gestão de avarias.

Pretendo continuar o desenvolvimento do website, por exemplo, melhorando o aspeto gráfico e adicionando mais possibilidades de gestão no painel de controlo do administrador. Contudo, nesta fase de desenvolvimento o projeto cumpre os objetivos propostos, refletindo as aprendizagens adquiridas na ação de formação Segurança no desenvolvimento de Software: codificação de password - BCrypt, verificação de critérios de input de texto, tratamento de dados em funções, utilização de lista de opção em vez de caixa de texto simples, divisão de processamento de tarefas em ficheiros dedicados.

A autenticação multifactor (MFA) com QR-Code não foi implementada devido aos recursos necessários.

Com a ajuda do SonarQube todos os erros e avisos foram corrigidos, à exceção um aviso já referido acima (figura 4).

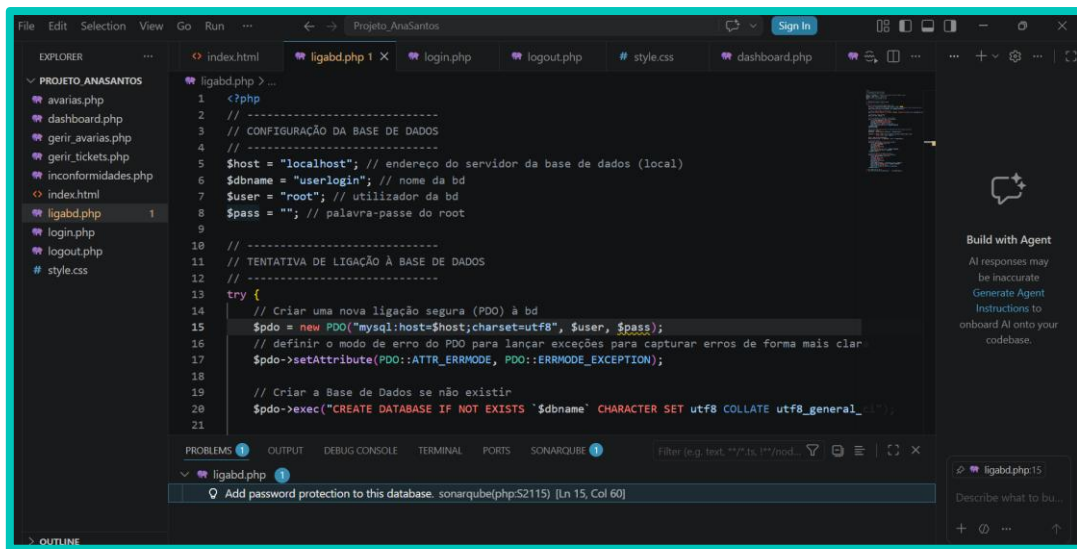


Figura 4